Anatomía de un LLM

Fundamentos matemáticos de los modelos de lenguaje

---

Fase VI · Diseño y construcción de optimizadores con IA

# Módulo 27

## Optimización geométrica y software 3D

Representaciones diferenciables, topología, simuladores, renderizado y pipelines 3D

---

Resumen del módulo

Edición de octubre de 2026

# Sobre este resumen

Este documento reúne lo indispensable de los apuntes del Módulo 27: el texto expositivo, todas las definiciones y todos los resultados (teoremas, proposiciones, lemas y corolarios), las tablas y las observaciones de contenido. Se han suprimido las demostraciones, los ejemplos, las lecturas, las figuras, los ejercicios y sus soluciones. La numeración de temas, secciones, enunciados y ecuaciones es la misma que en los apuntes, de modo que cualquier referencia puede buscarse allí; las que remiten a elementos suprimidos (un ejemplo, una figura, una ecuación de una demostración) conservan su número en los apuntes.

## Notación

| Símbolo | Significado |
|---|---|
| $\theta$ | parámetros de diseño (forma, densidades, coeficientes) |
| $(V, F)$, $E$ | malla: vértices, caras (triángulos) y aristas; $\chi = \lvert V\rvert - \lvert E\rvert + \lvert F\rvert$ es la característica de Euler |
| $f : \mathbb{R}^3 \to \mathbb{R}$ | función implícita o de distancia con signo (SDF): $f < 0$ dentro, $f = 0$ en la superficie |
| $x_e \in [0,1]$, $\tilde{x}_e$ | densidad de diseño del elemento $e$ y densidad física (filtrada) |
| $p$, $E_0$, $E_{\min}$ | exponente de penalización SIMP, módulo del material sólido y módulo mínimo |
| $K$, $U$, $F$ | matriz de rigidez, desplazamientos y cargas, con $KU = F$ |
| $c = F^\top U$ | flexibilidad (*compliance*): trabajo de las cargas; cuanto menor, más rígida la pieza |
| $k_0$, $u_e$ | matriz de rigidez del elemento con $E = 1$ y desplazamientos de sus nodos |
| $H_{ei}$, $r_{\min}$ | pesos del filtro y radio del filtro |
| $R(u, \theta) = 0$ | ecuaciones discretas de un simulador; $\lambda$ es el vector adjunto |
| $\hat{s}$ | modelo sustituto de una respuesta simulada $s$ |
| $\sigma(\cdot)$, sig | densidad volumétrica (NeRF) y función sigmoide $\mathrm{sig}(z) = 1/(1 + e^{-z})$ |
| $\alpha_i$, $T_i$, $w_i$ | opacidad, transmitancia y peso de la muestra $i$ de un rayo |
| $Q$ | cuádrica de error de un vértice (matriz simétrica $4 \times 4$) |
| $\bar{v} = (v, 1)$ | coordenadas homogéneas de un punto $v \in \mathbb{R}^3$ |

# Índice

**Sobre este resumen** **ii**

**1 Representaciones 3D y su diferenciabilidad** **1**
1.1 El optimizador geométrico . . . 1
1.2 Representaciones explícitas: mallas, nubes de puntos y vóxeles . . . 1
1.3 Representaciones implícitas y funciones de distancia con signo . . . 2
1.4 Campos neuronales . . . 3
1.5 Diferenciabilidad y elección de la representación . . . 3
1.6 Caso de estudio: ajuste de una esfera a un escaneo parcial . . . 4

**2 Optimización de forma y de topología** **5**
2.1 El problema de flexibilidad mínima . . . 5
2.2 La interpolación SIMP . . . 5
2.3 Sensibilidades por el método adjunto . . . 6
2.4 Actualización por criterios de optimalidad . . . 6
2.5 Inestabilidades numéricas y filtros . . . 7
2.6 Caso de estudio: la viga MBB . . . 7
2.7 Fabricación aditiva y diseño generativo . . . 7

**3 Simuladores diferenciables y modelos sustitutos** **9**
3.1 Derivar a través de un simulador . . . 9
3.2 Diferenciación automática de simuladores . . . 9
3.3 Modelos sustitutos . . . 10
3.4 Caso de estudio: el perfil de una aleta disipadora . . . 10

**4 Renderizado diferenciable y reconstrucción 3D** **12**
4.1 Formación de la imagen y rasterización . . . 12
4.2 Rasterización suave . . . 12
4.3 Renderizado volumétrico y NeRF . . . 13
4.4 *Gaussian splatting* . . . 14
4.5 Caso de estudio: ajuste de una forma a siluetas . . . 14

**5 Optimización de pipelines de software 3D y videojuegos** **15**
5.1 El presupuesto de rendimiento como restricción . . . 15
5.2 Simplificación de mallas por error cuádrico . . . 15
5.3 Niveles de detalle y error en pantalla . . . 16
5.4 Selección de niveles de detalle con presupuesto . . . 16
5.5 Compresión de texturas . . . 17
5.6 Empaquetado UV . . . 17
5.7 Contenido procedural y automatización con Blender . . . 17

**6 IA generativa para 3D y validación de la geometría** **19**
6.1 Tres maneras de generar geometría . . . 19
6.2 Validación de la geometría . . . 20
6.3 Generar y verificar . . . 21
6.4 Caso de estudio: validación de mallas generadas . . . 21

**Bibliografía** **22**

Tema 1

# Representaciones 3D y su diferenciabilidad

Antes de optimizar una forma hay que decidir cómo se describe con números. La elección no es neutral: determina qué cambios de forma son posibles (por ejemplo, si pueden aparecer agujeros), cuánta memoria hace falta y, sobre todo, si la magnitud que se quiere optimizar es una función derivable de los parámetros. Este tema presenta el esquema común de todos los optimizadores del módulo y repasa las representaciones clásicas y neuronales siguiendo la clase «Classical 3D Geometry Representations» del curso CS348n de Stanford [4], el artículo de DeepSDF [30], el de *marching cubes* [27] y el de las codificaciones con tablas *hash* de Müller *et al.* [29].

## 1.1 El optimizador geométrico

**Definición 1.1 (Optimizador geométrico).** Sea $\Theta \subseteq \mathbb{R}^n$ un conjunto de parámetros de diseño. Un *optimizador geométrico* resuelve

$$\min_{\theta \in \Theta} \; J(\theta) = \mathcal{L}\big(S(G(\theta))\big) \quad \text{sujeto a} \quad g_j(\theta) \le 0, \; j = 1, \dots, m,$$

donde $G$ construye una geometría a partir de $\theta$, $S$ la evalúa (un simulador físico, que devuelve campos como desplazamientos o temperaturas, o un renderizador, que devuelve imágenes), $\mathcal{L}$ resume la evaluación en un número (flexibilidad, calor disipado, error frente a fotografías) y las $g_j$ expresan restricciones de fabricación, de volumen o de presupuesto.

El gradiente $\nabla_\theta J$ puede obtenerse de tres maneras. Las diferencias finitas (módulo 4, tema 6) necesitan $n + 1$ evaluaciones de $S$ y son inviables cuando $n$ es del orden de miles. El modo inverso de la diferenciación automática o, cuando $S$ resuelve un sistema de ecuaciones, el método adjunto (módulo 10, §5.4, y tema 2) calculan el gradiente completo con un coste comparable a una o dos evaluaciones, independientemente de $n$. Y cuando $S$ no es derivable o es demasiado caro, se aprende un sustituto $\hat{s}$ con unas decenas de evaluaciones y se optimiza sobre él (tema 3). En los tres casos, la primera condición es que la geometría $G(\theta)$ dependa de $\theta$ de forma suave, y eso depende de la representación.

## 1.2 Representaciones explícitas: mallas, nubes de puntos y vóxeles

**Definición 1.2 (Malla triangular).** Una *malla triangular* es un par $(V, F)$ formado por una lista de vértices $v_1, \dots, v_{|V|} \in \mathbb{R}^3$ y una lista de caras, cada una un triplete ordenado $(a, b, c)$ de índices de vértices. El orden de los índices define la orientación de la cara y su normal

$n_f = (v_b - v_a) \times (v_c - v_a)/\|(v_b - v_a) \times (v_c - v_a)\|$. El conjunto $E$ de aristas está formado por los pares no ordenados de vértices consecutivos de alguna cara. La malla es *cerrada* si cada arista pertenece exactamente a dos caras, y está *orientada de forma coherente* si cada arista compartida se recorre en sentidos opuestos en sus dos caras.

**Proposición 1.3.** *En una malla triangular cerrada, $2|E| = 3|F|$. En particular, el número de caras es par.*

La característica de Euler $\chi = |V| - |E| + |F|$ de una malla cerrada y conexa que no se corta a sí misma depende solo de su topología: $\chi = 2 - 2g$, donde $g$ es el género o número de «asas» (0 para una esfera, 1 para un toro). Este resultado de topología se admite sin demostración.

**Corolario 1.4.** *Si una malla triangular cerrada tiene $\chi = 2$ (topología de esfera), entonces $|F| = 2|V| - 4$ y $|E| = 3|V| - 6$.*

El corolario da una regla de memoria útil: una malla cerrada tiene aproximadamente el doble de caras que de vértices. Con posiciones en coma flotante de 32 bits (12 bytes por vértice) e índices enteros de 32 bits (12 bytes por cara), cuesta unos 36 bytes por vértice.

**Definición 1.5 (Nube de puntos y distancia de Chamfer).** Una *nube de puntos* es un conjunto finito $A \subset \mathbb{R}^3$, opcionalmente con normales. La *distancia de Chamfer* entre dos nubes $A$ y $B$ es

$$\mathrm{CD}(A, B) = \frac{1}{|A|} \sum_{a \in A} \min_{b \in B} \|a - b\|^2 + \frac{1}{|B|} \sum_{b \in B} \min_{a \in A} \|a - b\|^2.$$

La distancia de Chamfer es derivable respecto a las coordenadas de los puntos salvo en los empates del mínimo: el gradiente respecto a $a$ empuja $a$ hacia su vecino más próximo en $B$. Es la pérdida habitual para ajustar nubes de puntos [2]. Una *rejilla de vóxeles* de resolución $n$ guarda un valor (ocupación, densidad o distancia) en cada una de las $n^3$ celdas de una caja; es la representación más sencilla de derivar, porque cada parámetro es un valor de celda, pero su memoria crece con el cubo de la resolución.

**Proposición 1.6 (Volumen de una malla cerrada y su gradiente).** *Sea $(V, F)$ una malla triangular cerrada, orientada de forma coherente con normales hacia fuera, que encierra la región $\Omega$. Entonces*

$$\mathrm{Vol}(\Omega) = \frac{1}{6} \sum_{(a,b,c) \in F} \det(v_a, v_b, v_c) = \frac{1}{6} \sum_{(a,b,c) \in F} v_a \cdot (v_b \times v_c),$$

*y la derivada respecto a la posición de un vértice $v_i$ es $\nabla_{v_i} \mathrm{Vol} = \frac{1}{6} \sum v_b \times v_c$, sumando sobre las caras que contienen a $i$, escritas en el orden cíclico $(i, b, c)$.*

## 1.3 Representaciones implícitas y funciones de distancia con signo

**Definición 1.8 (Superficie implícita y SDF).** Una función continua $f : \mathbb{R}^3 \to \mathbb{R}$ define el sólido $\Omega = \{f \le 0\}$ y la superficie implícita $\{f = 0\}$. Se dice que $f$ es una *función de distancia con signo* (SDF) de $\Omega$ si $f(x) = -\operatorname{dist}(x, \partial\Omega)$ para $x \in \Omega$ y $f(x) = \operatorname{dist}(x, \partial\Omega)$ fuera de $\Omega$.

Por ejemplo, $f(x) = \|x - c\| - r$ es la SDF de la bola de centro $c$ y radio $r$, y en todo punto $x \neq c$ su gradiente $\nabla f = (x - c)/\|x - c\|$ tiene norma 1.

**Proposición 1.9 (Una SDF es 1-lipschitziana).** *Si $f$ es la SDF de $\Omega$, entonces $|f(x) - f(y)| \le \|x-y\|$ para todo $x, y$. En consecuencia, en los puntos en que $f$ es derivable, $\|\nabla f\| \le 1$.*

De hecho, una SDF cumple casi en todas partes la *ecuación eikonal* $\|\nabla f\| = 1$, y su gradiente sobre la superficie es la normal exterior. Las operaciones booleanas son inmediatas: si $f_1$ y $f_2$ describen $\Omega_1$ y $\Omega_2$, entonces $\min(f_1, f_2)$ describe la unión, $\max(f_1, f_2)$ la intersección y $\max(f_1, -f_2)$ la diferencia $\Omega_1 \setminus \Omega_2$. Los conjuntos son exactos, aunque los valores resultantes pueden ser solo cotas de la distancia (por ejemplo, dentro de una unión). Esta robustez de las operaciones es el argumento del modelado implícito que usan herramientas comerciales como nTop, que publica que sus operaciones booleanas, desfases y redondeos no fallan, y que modula celosías y espesores con campos escalares procedentes de simulaciones [45].

Para fabricar o renderizar con rasterización hay que convertir $f$ en una malla. El algoritmo de *marching cubes* [27] evalúa $f$ en los nodos de una rejilla y, en cada arista con cambio de signo entre $f_0 < 0$ y $f_1 > 0$, coloca un vértice por interpolación lineal en la fracción $t = f_0/(f_0 - f_1)$ de la arista; una tabla indexada por el patrón de signos de los ocho vértices del cubo da los triángulos. El coste es una evaluación de $f$ por nodo, $n^3$ en total, y el error de posición es del orden del paso de la rejilla.

## 1.4 Campos neuronales

Un *campo neuronal* representa $f$ con una red, $f_\varphi(x)$, típicamente un perceptrón multicapa (módulo 9). DeepSDF [30] ajusta una red de ocho capas de 512 unidades a valores de SDF muestreados con la pérdida $L^1$ recortada

$$\ell\big(f_\varphi(x), s\big) = \big|\mathrm{clip}(f_\varphi(x), -\delta, \delta) - \mathrm{clip}(s, -\delta, \delta)\big|,$$

con $\delta = 0{,}1$, que concentra la capacidad cerca de la superficie. Para representar una familia de formas, DeepSDF asigna a cada forma un código latente $z_i$ que se optimiza junto con los pesos, con un término $\|z_i\|^2/\sigma^2$ que procede de un prior gaussiano (un *autodecodificador*); a una forma nueva se le asigna su código optimizando solo $z$ con los pesos congelados. La analogía con un LLM es directa: los códigos $z_i$ desempeñan el papel de las filas de la matriz de embeddings (módulo 12, §4.1), que también son parámetros libres aprendidos por descenso de gradiente.

Cuando no se dispone de valores de SDF sino solo de puntos de la superficie, se impone $f_\varphi = 0$ en ellos y se añade la regularización eikonal $\mathbb{E}_x\big[(\|\nabla_x f_\varphi(x)\| - 1)^2\big]$, cuyo gradiente respecto a $x$ se obtiene por diferenciación automática [5]. Los perceptrones con entradas $(x, y, z)$ aprenden mal los detalles finos; dos remedios son la codificación posicional sinusoidal (tema 4), análoga a la del Transformer (módulo 14, §4.2), y las rejillas multirresolución de vectores entrenables indexadas con funciones *hash*, con las que Müller *et al.* entrenan campos neuronales de calidad en segundos [29].

## 1.5 Diferenciabilidad y elección de la representación

El cuadro 1.1 resume el compromiso. Una malla con conectividad fija es compacta y es lo que consumen los motores y las impresoras, pero su optimización no puede abrir agujeros; las representaciones volumétricas (densidades, SDF, campos) sí pueden, y por eso la optimización de topología (tema 2) trabaja con densidades por elemento. En todas ellas la derivabilidad

| Representación | Parámetros derivables | Cambios de topología | Memoria |
|---|---|---|---|
| Malla | posiciones de los vértices (conectividad fija) | no, salvo remallado | 4,1 MB |
| Nube de puntos | coordenadas de los puntos | no aplica (sin superficie) | — |
| Vóxeles (ocupación) | valor de cada celda | sí | 0,29–2,3 MB |
| SDF en rejilla | valor de cada nodo | sí | 9,2 MB |
| Campo neuronal | pesos de la red | sí | 7,4 MB |

Cuadro 1.1: Comparación para un soporte que cabe en una caja de $120 \times 60 \times 40$ mm con resolución de 0,5 mm. Vóxeles: $240 \cdot 120 \cdot 80 = 2\,304\,000$ celdas (1 bit o 1 byte por celda). SDF: 4 bytes por nodo. Malla: superficie de la caja 28 800 mm² en triángulos rectángulos de catetos 0,5 mm, 230 400 caras y 115 200 vértices. Red: 8 capas de 512 unidades, $1{,}84 \cdot 10^6$ parámetros de 4 bytes.

se refiere a la magnitud que se evalúa: el volumen de una malla es un polinomio en las coordenadas (proposición 1.6), pero la imagen rasterizada de esa misma malla es constante a trozos (tema 4).

## 1.6 Caso de estudio: ajuste de una esfera a un escaneo parcial

| Herramienta | Para qué sirve | Licencia | Cuándo elegirla |
|---|---|---|---|
| trimesh | cargar, reparar y medir mallas (volumen, estanqueidad) | MIT | validación y utilidades en Python puro |
| Open3D | nubes de puntos, registro, reconstrucción | MIT | datos de escáner y visualización |
| PyMeshLab | filtros de MeshLab (remallado, simplificación) | GPL-3.0 | procesado por lotes con algoritmos maduros |
| libigl | geometría discreta (laplacianos, parametrización) | MPL-2.0 | investigación y prototipos de procesado de geometría |
| scikit-image | *marching cubes* sobre rejillas | BSD-3 | extraer mallas de SDF o densidades |
| PyTorch3D | mallas y nubes por lotes, pérdidas y renderizado derivables | BSD | ajuste de formas con gradientes en PyTorch |
| Kaolin (NVIDIA) | operadores 3D derivables y conversiones | Apache-2.0 | ecosistema NVIDIA con GPU |

Cuadro 1.2: Tecnologías del tema 1. Licencias según los metadatos publicados por cada proyecto.

Tema 2

# Optimización de forma y de topología

La optimización de topología responde a una pregunta de diseño mecánico: dado un volumen de trabajo, unos apoyos, unas cargas y una cantidad de material, ¿dónde se pone el material para que la pieza sea lo más rígida posible? A diferencia de la optimización de forma, que mueve una frontera ya existente, aquí se permite que aparezcan y desaparezcan agujeros. Este tema sigue el capítulo 1 de Bendsøe y Sigmund [10], el artículo de Bendsøe de 1989 que formuló el problema como distribución de material [14], los códigos didácticos de 99 y 88 líneas [36, 13], el estudio de inestabilidades de Sigmund y Petersson [37] y el análisis de filtros de Bourdin [15]. El método adjunto se presenta como en las notas de cálculo matricial del MIT [1, §6.3].

## 2.1 El problema de flexibilidad mínima

Se discretiza el dominio de diseño en $N$ elementos finitos cuadrados. El elemento $e$ tiene una densidad $x_e \in [0,1]$ (0 vacío, 1 material) y una matriz de rigidez $E_e\,k_0$, donde $k_0 \in \mathbb{R}^{8\times 8}$ es la del elemento bilineal con módulo unidad. El ensamblado da la matriz global $K(x) = \sum_e E_e(x_e)\,P_e^{\top} k_0 P_e$, simétrica y definida positiva una vez impuestos los apoyos, donde $P_e$ extrae los ocho grados de libertad del elemento.

**Definición 2.1 (Flexibilidad mínima con restricción de volumen).** Dadas las cargas $F$ y una fracción de volumen $f_V \in (0,1)$, el problema es

$$\min_{x\in[0,1]^N} \; c(x) = F^{\top}U(x) \quad \text{sujeto a} \quad K(x)\,U(x) = F, \qquad \frac{1}{N}\sum_{e=1}^{N} x_e = f_V .$$

La *flexibilidad* $c = F^{\top}U = U^{\top}KU$ es el trabajo de las cargas exteriores, igual al doble de la energía elástica almacenada; minimizarla equivale a maximizar la rigidez global.

Si las densidades solo pudieran valer 0 o 1, el problema sería combinatorio, con $2^N$ diseños. La relajación a $[0,1]$ lo vuelve continuo y derivable, pero entonces hay que conseguir que el óptimo no esté lleno de material «gris» de densidad intermedia, que no se puede fabricar.

## 2.2 La interpolación SIMP

**Definición 2.2 (SIMP modificado).** El método SIMP (*solid isotropic material with penalization*) interpola el módulo de Young como

$$E_e(x_e) = E_{\min} + x_e^{\,p}\,(E_0 - E_{\min}), \qquad p > 1,$$

con $E_{\min} > 0$ pequeño (por ejemplo $10^{-9}E_0$) para que $K$ no sea singular cuando hay elementos vacíos [13, §2.1].

La penalización funciona porque el volumen es lineal en $x_e$ y la rigidez no. Con $p = 3$, un elemento a densidad 0,5 consume la mitad del material pero aporta solo $0{,}5^3 = 0{,}125$ de la rigidez: la relación rigidez/volumen es $x_e^{p-1} = 0{,}25$, cuatro veces peor que la del material sólido. El optimizador aprende así a evitar los grises. El precio es la pérdida de convexidad.

**Teorema 2.3 (Convexidad con $p = 1$).** *Si $E_e(x_e) = E_{\min} + x_e(E_0 - E_{\min})$, es decir, con $p = 1$, la flexibilidad $c(x) = F^\top K(x)^{-1} F$ es una función convexa de $x$ en $[0,1]^N$.*

Con $p = 1$ (la «chapa de espesor variable») el problema es convexo y su óptimo global es fácil de alcanzar, pero contiene grandes zonas grises. Con $p = 3$, el valor estándar, desaparece la convexidad y el resultado depende del punto de partida y de los parámetros; el filtrado y las estrategias de continuación en $p$ mitigan el problema [37].

## 2.3 Sensibilidades por el método adjunto

**Teorema 2.4 (Método adjunto para ecuaciones de equilibrio).** *Sea $J(x) = g\big(x, U(x)\big)$ con $K(x)U = F$, $K(x)$ invertible y derivable, y $F$ independiente de $x$. Sea $\lambda$ la solución del* problema adjunto $K(x)^\top \lambda = \big(\partial g/\partial U\big)^\top$. *Entonces*

$$\frac{dJ}{dx_e} = \frac{\partial g}{\partial x_e} - \lambda^\top \frac{\partial K}{\partial x_e} U.$$

La demostración es la asociatividad del producto de jacobianos que distingue el modo inverso del directo (módulo 10, §4.2): multiplicar primero $\frac{\partial g}{\partial U} K^{-1}$ cuesta una sola resolución, sea cual sea el número $N$ de variables, mientras que calcular cada $dU/dx_e$ costaría $N$ resoluciones. El vector $\lambda$ es el multiplicador de Lagrange de la restricción $KU = F$ (módulo 10, §5.4).

**Corolario 2.5 (La flexibilidad es autoadjunta).** *Para $c = F^\top U$ se tiene $\lambda = U$ y*

$$\frac{\partial c}{\partial x_e} = -U^\top \frac{\partial K}{\partial x_e} U = -p\, x_e^{p-1} (E_0 - E_{\min})\, u_e^\top k_0 u_e \;\le 0.$$

El signo tiene una lectura física: añadir material nunca hace la estructura más flexible. Calcular el gradiente completo no cuesta ninguna resolución adicional; basta con la energía $u_e^\top k_0 u_e$ de cada elemento.

## 2.4 Actualización por criterios de optimalidad

Con $V(x) = \sum_e v_e x_e$ (en la malla regular, $v_e = 1$), la lagrangiana del problema es $c(x) + \Lambda\big(V(x) - N f_V\big)$ más los términos de las cotas. Para un elemento con $0 < x_e < 1$, la condición de estacionariedad (0.7, §5.4) es $-\partial c/\partial x_e = \Lambda\, v_e$, es decir,

$$B_e := \frac{-\partial c/\partial x_e}{\Lambda\, \partial V/\partial x_e} = 1.$$

La actualización por criterios de optimalidad (OC) convierte esta condición en una iteración de punto fijo: si $B_e > 1$, el elemento rinde más que el precio $\Lambda$ del material y conviene aumentar $x_e$; si $B_e < 1$, disminuirla.

**Definición 2.7 (Actualización OC).** Con amortiguamiento $\eta = 1/2$ y límite de movimiento $m$ (0,2 en [13]),

$$x_e^{\text{nuevo}} = \text{clip}\Big(x_e B_e^{\eta},\ \max(0, x_e - m),\ \min(1, x_e + m)\Big),$$

donde $\Lambda$ se elige por bisección para que $x^{\text{nuevo}}$ cumpla la restricción de volumen.

**Proposición 2.8.** *Si $\partial c/\partial x_e < 0$ para todo $e$, el volumen $V\big(x^{nuevo}(\Lambda)\big)$ es una función continua y no creciente de $\Lambda > 0$; por tanto, la bisección encuentra un $\Lambda$ que cumple la restricción siempre que esta sea alcanzable dentro de los límites de movimiento.*

## 2.5 Inestabilidades numéricas y filtros

Sin regularización, SIMP produce dos patologías bien documentadas [37]: *tableros de ajedrez*, zonas de elementos sólidos y vacíos alternados que el elemento bilineal considera artificialmente rígidas, y *dependencia de la malla*: al refinar aparecen cada vez más barras finas, porque el problema continuo carece de solución con microestructura acotada. La solución estándar es imponer una escala de longitud mínima mediante un filtro de radio $r_{\min}$, con pesos

$$H_{ei} = \max\big(0,\ r_{\min} - \text{dist}(e, i)\big),$$

donde la distancia se mide entre centros de elementos.

**Definición 2.9 (Filtros de sensibilidades y de densidades).** El *filtro de sensibilidades* [36, 13] sustituye las derivadas por un promedio ponderado,

$$\widehat{\frac{\partial c}{\partial x_e}} = \frac{1}{\max(\gamma, x_e)\sum_i H_{ei}} \sum_i H_{ei}\, x_i \frac{\partial c}{\partial x_i}, \qquad \gamma = 10^{-3}.$$

El *filtro de densidades* [15] define densidades físicas $\tilde{x}_e = \sum_i H_{ei} x_i / \sum_i H_{ei}$, con las que se calcula la rigidez; las sensibilidades respecto a las variables de diseño se obtienen por la regla de la cadena,

$$\frac{\partial c}{\partial x_j} = \sum_e \frac{\partial c}{\partial \tilde{x}_e}\frac{\partial \tilde{x}_e}{\partial x_j} = \sum_e \frac{H_{ej}}{\sum_i H_{ei}} \frac{\partial c}{\partial \tilde{x}_e}.$$

El filtro de densidades tiene una justificación matemática: Bourdin demuestra que el problema filtrado tiene solución y que las aproximaciones por elementos finitos convergen [15]. El filtro de sensibilidades es heurístico (el gradiente filtrado no es el de ninguna función), pero funciona muy bien en la práctica. En ambos casos, el diseño que se presenta es el de las densidades físicas $\tilde{x}$, no el de las variables $x$ [13, §2.3]. La proyección de Heaviside, $\bar{x} = 1 - e^{-\beta\tilde{x}} + \tilde{x}e^{-\beta}$ con $\beta$ creciente, elimina los grises que el filtro de densidades deja en los bordes.

## 2.6 Caso de estudio: la viga MBB

## 2.7 Fabricación aditiva y diseño generativo

Un diseño optimizado solo es útil si se puede fabricar. Las restricciones más comunes son el tamaño mínimo de las barras y de los huecos, que el radio del filtro controla; la dirección de fabricación, porque en fabricación aditiva las superficies que miran hacia abajo con poca

inclinación necesitan soportes (tema 6); y, para mecanizado o fundición, la accesibilidad de la herramienta y el desmoldeo. Langelaar propuso un filtro que, capa a capa, solo permite material que esté sostenido por material de la capa inferior, de modo que el resultado es imprimible sin soportes [22]. Tras optimizar, el flujo habitual es umbralizar $\tilde{x}$, extraer la superficie (*marching cubes* o *squares*, tema 1), suavizarla, reconstruir una geometría CAD y *volver a verificar* con un análisis de elementos finitos más fino, incluyendo tensiones, que el problema de flexibilidad no controla.

Los programas comerciales de *diseño generativo* empaquetan este flujo. La documentación pública de Autodesk Fusion describe métodos de fabricación seleccionables (sin restricciones, aditiva con dirección de impresión, fresado de 2,5, 3 y 5 ejes, corte en 2 ejes y fundición con espesor mínimo, ángulo de desmoldeo y dirección de expulsión), cada uno de los cuales genera su propio conjunto de resultados [39]. El caso publicado más citado es el soporte de cinturón de un asiento de General Motors: según Autodesk, el diseño generativo consolidó ocho piezas en una sola de acero inoxidable, un 40 % más ligera y un 20 % más resistente que la anterior [38]. nTop combina el modelado implícito con resultados de optimización de topología y con campos que modulan celosías y espesores [45]. No se han publicado los algoritmos internos de estos productos.

| Herramienta | Para qué sirve | Licencia o tipo | Cuándo elegirla |
|---|---|---|---|
| top88 y variantes (DTU) | códigos didácticos SIMP en MATLAB | descarga gratuita (DTU) | aprender y prototipar en 2D |
| numpy + scipy.sparse | montaje y resolución de elementos finitos | BSD | reproducir top88 en Python (cuaderno) |
| NLopt (MMA) | optimización con muchas restricciones | MIT | sustituir OC cuando hay varias restricciones |
| dolfin-adjoint | adjuntos automáticos sobre FEniCS | LGPL | EDP generales en 2D y 3D sin derivar a mano |
| Autodesk Fusion | diseño generativo con restricciones de fabricación | comercial | ingeniería de producto con CAD integrado |
| nTop | modelado implícito, celosías, campos | comercial | piezas para fabricación aditiva con celosías |

Cuadro 2.1: Tecnologías del tema 2.

Tema 3

# Simuladores diferenciables y modelos sustitutos

El tema 2 obtuvo el gradiente de un simulador de elementos finitos derivando a mano sus ecuaciones. Este tema generaliza la idea a cualquier simulador, estacionario o temporal, y presenta la alternativa cuando derivar no es posible o el simulador es demasiado caro: aprender un modelo sustituto con pocas simulaciones y optimizar sobre él. Se siguen las notas de S. G. Johnson sobre adjuntos para recurrencias [7], las de cálculo matricial del MIT [1, §6.3], los capítulos de modelos sustitutos de Kochenderfer y Wheeler [12], y los artículos de DiffTaichi [16], las redes informadas por la física [33], el operador neuronal de Fourier [25] y MeshGraphNets [31]. La física de la aleta procede del capítulo 3 de Incropera *et al.* [11].

## 3.1 Derivar a través de un simulador

Casi todo simulador discreto resuelve un sistema de ecuaciones $R(u,\theta)=0$, donde $u\in\mathbb{R}^m$ es el estado (temperaturas, desplazamientos, velocidades) y $\theta\in\mathbb{R}^n$ son los parámetros de diseño.

**Teorema 3.1 (Sensibilidad por la función implícita y el adjunto).** *Sean $R:\mathbb{R}^m\times\mathbb{R}^n\to\mathbb{R}^m$ y $J(\theta)=g\big(u(\theta),\theta\big)$ funciones continuamente derivables, con $R(u(\theta),\theta)=0$ y $\partial R/\partial u$ invertible en la solución. Entonces*

$$\frac{du}{d\theta}=-\Big(\frac{\partial R}{\partial u}\Big)^{-1}\frac{\partial R}{\partial \theta},\qquad \frac{dJ}{d\theta}=\frac{\partial g}{\partial\theta}-\lambda^{\top}\frac{\partial R}{\partial\theta},\quad \text{con}\quad \Big(\frac{\partial R}{\partial u}\Big)^{\top}\lambda=\Big(\frac{\partial g}{\partial u}\Big)^{\top}.$$

El teorema 2.4 es el caso $R=K(x)U-F$. El coste del adjunto es una resolución con la matriz transpuesta, que en problemas lineales puede reutilizar la factorización ya calculada; el de la sensibilidad directa, $n$ resoluciones. Para simuladores temporales, $u_{k+1}=\Phi(u_k,\theta)$, el mismo razonamiento da una recurrencia adjunta que se recorre hacia atrás en el tiempo [7]:

$$\lambda_N=\Big(\frac{\partial g}{\partial u_N}\Big)^{\top},\qquad \lambda_{k-1}=\Big(\frac{\partial \Phi}{\partial u}\Big)_k^{\top}\lambda_k,\qquad \frac{dg}{d\theta}=\frac{\partial g}{\partial\theta}+\sum_{k=1}^{N}\lambda_k^{\top}\frac{\partial\Phi}{\partial\theta}\Big|_{u_{k-1}}.$$

Es la retropropagación a través del tiempo de una red recurrente (módulo 12, §5.3), con los mismos problemas: hay que guardar todos los estados, lo que se alivia con el recálculo por puntos de control (módulo 10, §5.2), y en dinámicas caóticas o con muchos pasos el gradiente puede explotar.

## 3.2 Diferenciación automática de simuladores

En lugar de derivar a mano, el simulador se puede escribir en un lenguaje que propague derivadas. JAX transforma programas de Python y NumPy (derivación, vectorización y compilación

para GPU y TPU) [41]; NVIDIA Warp compila núcleos de Python para GPU que son derivables [47]; Taichi, con su extensión DiffTaichi, permite escribir simuladores físicos derivables, y sus autores informan de que su simulador elástico necesita unas 4,2 veces menos líneas que una versión CUDA escrita a mano, con velocidad similar, y es 188 veces más rápido que la implementación en TensorFlow [16]. PyTorch también sirve cuando el simulador se expresa con operaciones de tensores, como en el caso de estudio de este tema.

Tres advertencias limitan el enfoque. Los eventos discretos (contactos, cambios de malla, condicionales sobre el estado) producen funciones discontinuas cuya derivada automática es correcta pero inútil, porque vale cero o no ve el salto. Las derivadas de un resolvedor iterativo no deben calcularse a través de sus iteraciones, sino con el teorema 3.1 en la solución convergida. Y el coste de memoria del modo inverso crece con el número de pasos de tiempo.

## 3.3 Modelos sustitutos

**Definición 3.2 (Modelo sustituto).** Sea $s : \Theta \to \mathbb{R}$ una respuesta cara de evaluar (el resultado de un simulador). Un *modelo sustituto* es una función $\hat{s}_\varphi$ barata, ajustada con un conjunto de diseño $\{(\theta_i, s(\theta_i))\}_{i=1}^{M}$, con la que se sustituye a $s$ dentro de la optimización.

Los sustitutos clásicos son las superficies de respuesta polinómicas ajustadas por mínimos cuadrados y los procesos gaussianos, que además estiman su incertidumbre [12]. Los neuronales cubren desde un perceptrón que aproxima una respuesta escalar hasta modelos que aproximan campos completos: las redes informadas por la física (PINN) minimizan el residuo de la ecuación diferencial en puntos de colocación, calculando las derivadas de la red por diferenciación automática [33]; los operadores neuronales de Fourier aprenden la aplicación entre funciones de entrada y de salida, y sus autores informan de aceleraciones de hasta tres órdenes de magnitud frente a resolvedores clásicos [25]; MeshGraphNets aprende la dinámica sobre la malla de simulación con una red de grafos [31]. Bibliotecas como NVIDIA PhysicsNeMo (antes Modulus) agrupan estas arquitecturas para construir sustitutos de CFD, mecánica estructural o análisis térmico [46].

El diseño de experimentos importa tanto como el modelo. El muestreo por hipercubo latino divide el rango de cada variable en $M$ intervalos y toma exactamente una muestra por intervalo, de modo que sus proyecciones sobre cada eje quedan uniformemente repartidas [12].

**Proposición 3.3 (Error del óptimo de un sustituto).** *Sea $\Theta$ compacto, $s$ y $\hat{s}$ continuas y $\sup_{\theta\in\Theta}|\hat{s}(\theta) - s(\theta)| \le \varepsilon$. Si $\theta^\star$ minimiza $s$ y $\hat{\theta}$ minimiza $\hat{s}$ en $\Theta$, entonces*

$$s(\theta^\star) \le s(\hat{\theta}) \le s(\theta^\star) + 2\varepsilon.$$

La proposición es tranquilizadora y peligrosa a la vez: la garantía exige un error *uniforme* en todo $\Theta$, mientras que la validación solo mide el error en unos puntos. El optimizador busca precisamente las zonas en que $\hat{s}$ es optimista, que suelen ser aquellas en que hay pocos datos. Es el mismo fenómeno que la sobreoptimización de un modelo de recompensa (módulo 21, §6.4). Por eso todo diseño propuesto por un sustituto se verifica con el simulador y, si hay presupuesto, se añade a los datos y se reajusta el modelo.

## 3.4 Caso de estudio: el perfil de una aleta disipadora

| Herramienta | Para qué sirve | Licencia | Cuándo elegirla |
|---|---|---|---|
| JAX | derivación, vectorización y compilación de código NumPy | Apache-2.0 | simuladores nuevos escritos en estilo funcional |
| NVIDIA Warp | núcleos de simulación derivables en GPU | Apache-2.0 | física de partículas, telas o sólidos en GPU |
| Taichi (DiffTaichi) | lenguaje para simulación física derivable | Apache-2.0 | simuladores de partículas y rejillas de alto rendimiento |
| PyTorch | tensores y diferenciación automática | BSD-3 (con componentes de otras licencias) | simuladores pequeños unidos a redes neuronales |
| NVIDIA PhysicsNeMo | PINN, operadores neuronales y GNN para física | Apache-2.0 | sustitutos de CFD o de estructuras a escala industrial |
| scikit-learn | MLP, procesos gaussianos, validación | BSD-3 | sustitutos de pocas variables con pocos datos |
| dolfin-adjoint | adjuntos automáticos de modelos FEniCS | LGPL | EDP generales con elementos finitos |

Cuadro 3.1: Tecnologías del tema 3. Licencias según los metadatos publicados por cada proyecto.

Tema 4

# Renderizado diferenciable y reconstrucción 3D

Reconstruir un objeto a partir de fotografías es un problema inverso: se busca la geometría (y la apariencia) cuyas imágenes renderizadas coinciden con las observadas. Si el renderizador es derivable, el problema se resuelve por descenso de gradiente, igual que el entrenamiento de una red. Este tema explica por qué la rasterización clásica no es derivable, cómo la suavizan SoftRas [26] y las bibliotecas actuales [34, 21, 42], y cómo NeRF [28] y el *Gaussian splatting* [20] convierten la reconstrucción de escenas en una optimización. Se apoya en las clases «Graphics Pipeline and Rasterization» de MIT 6.837 [6], en la clase de renderizado neuronal de CS348n [4] y en las tareas 2 y 3 del curso 16-825 de Carnegie Mellon [2].

## 4.1 Formación de la imagen y rasterización

Una cámara estenopeica con matriz de rotación $W$, centro $o$ y distancia focal $\phi$ (en píxeles) proyecta el punto $X$ en el píxel $\big(\phi\, X_c^{(1)}/X_c^{(3)},\ \phi\, X_c^{(2)}/X_c^{(3)}\big)$, con $X_c = W(X - o)$. La rasterización recorre los triángulos proyectados y asigna a cada píxel el triángulo más cercano que lo cubre (*z-buffer*) [6]. La silueta de una malla es la función indicadora de los píxeles cubiertos.

**Proposición 4.1 (La rasterización tiene gradiente nulo casi en todas partes).** *Sea $I_p(\theta) \in \{0, 1\}$ el valor de la silueta en el píxel $p$ (muestreado en su centro) como función de las coordenadas $\theta$ de los vértices. Entonces $I_p$ es constante a trozos: en todo $\theta$ en que ningún borde de triángulo proyectado pasa exactamente por el centro de $p$, $\nabla_\theta I_p = 0$.*

Una pérdida que compare siluetas rasterizadas tiene, por tanto, gradiente nulo salvo en un conjunto de medida nula, donde no existe: el descenso de gradiente no recibe información. Hay dos soluciones: suavizar el renderizador, o calcular explícitamente la contribución de los bordes (como hacen los métodos de antialias derivable de nvdiffrast [21] y los renderizadores físicos como Mitsuba 3 [42]).

## 4.2 Rasterización suave

**Definición 4.2 (Mapa de probabilidad de SoftRas).** Para el píxel $p_i$ y el triángulo proyectado $f_j$, SoftRas [26, §3.2] define

$$D_j^i = \operatorname{sig}\Big(\delta_j^i \,\frac{d^2(i,j)}{\sigma}\Big),$$

donde $d(i, j)$ es la distancia del píxel al borde del triángulo, $\delta_j^i = +1$ si el píxel está dentro y $-1$ si está fuera, y $\sigma > 0$ controla la nitidez. La silueta suave se obtiene agregando los

triángulos como sucesos independientes [26, §3.3],

$$I_s^i = 1 - \prod_j \big(1 - D_j^i\big).$$

Cuando $\sigma \to 0$, $D_j^i$ tiende a la indicadora del triángulo y se recupera la rasterización clásica; con $\sigma > 0$, todos los triángulos cercanos reciben gradiente, incluso los ocultos y los vértices alejados del borde. El valor por defecto en el artículo es $\sigma = 10^{-4}$ en coordenadas de imagen normalizadas. La pérdida de siluetas más usada es $1 - \mathrm{IoU}$ suave, con $\mathrm{IoU}(S,T) = \sum_i S_i T_i / \sum_i (S_i + T_i - S_i T_i)$ [26, §4.1]. El compromiso es claro: un $\sigma$ grande da gradientes en una región amplia, útiles al principio, pero sesga la silueta (la «hincha»); un $\sigma$ pequeño es fiel pero casi no da gradiente. De ahí la práctica de *recocer* $\sigma$ de grande a pequeño, que se ve en el caso de estudio.

Las bibliotecas actuales implementan estas ideas con eficiencia. PyTorch3D proporciona estructuras de mallas y nubes por lotes, pérdidas 3D y un renderizador derivable modular [34]. nvdiffrast ofrece primitivas de alto rendimiento (rasterización, interpolación de atributos, filtrado de texturas y antialias) que aprovechan el hardware gráfico y se integran en PyTorch o TensorFlow [21]. Mitsuba 3, construido sobre el compilador Dr.Jit, calcula derivadas de simulaciones completas de transporte de luz respecto a la cámara, la geometría, los materiales o las texturas [42].

## 4.3 Renderizado volumétrico y NeRF

Un campo de radiancia neuronal (NeRF) representa una escena con una red que, para cada punto $X$ y dirección de vista $d$, devuelve una densidad $\sigma(X) \geq 0$ y un color $c(X,d)$ [28]. El color de un rayo $r(t) = o + td$ es

$$C(r) = \int_{t_n}^{t_f} T(t)\,\sigma(r(t))\,c(r(t),d)\,dt, \qquad T(t) = \exp\Big(-\int_{t_n}^{t} \sigma(r(u))\,du\Big),$$

donde $T(t)$ es la transmitancia, la probabilidad de que la luz recorra el rayo hasta $t$ sin ser absorbida. Con muestras $t_1 < \cdots < t_N$ y $\delta_i = t_{i+1} - t_i$, la cuadratura de NeRF [28, §4] es

$$\hat{C}(r) = \sum_{i=1}^{N} w_i\, c_i, \qquad w_i = T_i\,\alpha_i, \qquad \alpha_i = 1 - e^{-\sigma_i \delta_i}, \qquad T_i = \exp\Big(-\sum_{j<i} \sigma_j \delta_j\Big) = \prod_{j<i} (1-\alpha_j).$$

**Proposición 4.3 (Los pesos del rayo).** *Con $\sigma_i \geq 0$, los pesos cumplen $w_i \geq 0$ y $\sum_{i=1}^{N} w_i = 1 - T_{N+1} \leq 1$, donde $T_{N+1} = \prod_{j \leq N}(1-\alpha_j)$ es la transmitancia residual.*

Los pesos forman, pues, una distribución de probabilidad (completada con el fondo, de peso $T_{N+1}$) sobre el punto del rayo en que la luz se emite; la profundidad esperada es $\sum_i w_i t_i$. El renderizado es derivable respecto a $\sigma_i$ y $c_i$, y por tanto respecto a los pesos de la red. NeRF aplica a las coordenadas una codificación posicional $\gamma(p) = \big(\sin(2^0\pi p), \cos(2^0\pi p), \ldots, \sin(2^{L-1}\pi p), \cos(2^{L-1}\pi p)\big)$ con $L = 10$ para la posición y $L = 4$ para la dirección, una red de 8 capas de 256 unidades, y minimiza el error cuadrático entre colores renderizados y observados con lotes de 4096 rayos; cada escena necesita entre $10^5$ y $3 \cdot 10^5$ iteraciones, uno o dos días en una GPU V100 [28, §5].

## 4.4 *Gaussian splatting*

El *Gaussian splatting* 3D [20] sustituye la red por un conjunto explícito de gaussianas anisótropas, cada una con media $\mu$, covarianza $\Sigma$, opacidad $o$ y color dependiente de la vista (coeficientes de armónicos esféricos). Para que la covarianza siga siendo válida durante la optimización se parametriza como

$$\Sigma = R\,S\,S^{\top}R^{\top},$$

con $R$ la rotación de un cuaternión normalizado y $S$ diagonal de escalas [20, §4]. Cada gaussiana se proyecta a la imagen con la covarianza $\Sigma' = JW\Sigma W^{\top}J^{\top}$, donde $J$ es el jacobiano de la aproximación afín de la proyección, y las gaussianas que cubren un píxel, ordenadas por profundidad, se componen con la misma fórmula que NeRF:

$$C = \sum_i c_i\,\alpha_i \prod_{j<i}(1-\alpha_j), \qquad \alpha_i = o_i \exp\!\Big(-\tfrac{1}{2}(p-\mu_i')^{\top}\Sigma_i'^{-1}(p-\mu_i')\Big).$$

La pérdida combina $L^1$ y D-SSIM con peso $\lambda = 0{,}2$; un control adaptativo clona o divide gaussianas donde el gradiente de posición es grande y elimina las casi transparentes, y un rasterizador por teselas de $16\times16$ píxeles con ordenación por radix en GPU permite renderizar en tiempo real (al menos 30 fotogramas por segundo a 1080p) escenas de uno a cinco millones de gaussianas [20]. Frente a NeRF, el *splatting* cambia una representación implícita y compacta, pero lenta de muestrear, por una explícita, rápida y más voluminosa en memoria.

## 4.5 Caso de estudio: ajuste de una forma a siluetas

| Herramienta | Para qué sirve | Licencia | Cuándo elegirla |
|---|---|---|---|
| PyTorch3D | rasterización suave de mallas y nubes, pérdidas 3D | BSD | ajuste de mallas e investigación en PyTorch |
| nvdiffrast | primitivas de rasterización derivable de alto rendimiento | licencia de código de NVIDIA | mallas grandes, texturas y antialias en GPU NVIDIA |
| Mitsuba 3 | renderizado físico (transporte de luz) derivable | BSD-3 (con cláusula adicional) | materiales, iluminación indirecta, medios participantes |
| Kaolin | conversiones y renderizado derivable | Apache-2.0 | ecosistema NVIDIA |
| nerfstudio | entrenamiento modular de NeRF | Apache-2.0 | reconstrucción de escenas con varias variantes de NeRF |
| gsplat | rasterizador de gaussianas | Apache-2.0 | *Gaussian splatting* con licencia permisiva |
| 3DGS original (Inria) | implementación de referencia de [20] | solo uso no comercial | investigación y comparación |

Cuadro 4.1: Tecnologías del tema 4, con las licencias publicadas en sus repositorios.

Tema 5

# Optimización de pipelines de software 3D y videojuegos

En producción, la geometría no se optimiza solo para que una pieza sea rígida o una reconstrucción fiel, sino para que una escena quepa en un presupuesto: milisegundos por fotograma, megabytes de memoria de vídeo, tamaño de descarga. Este tema formula como problemas de optimización las decisiones habituales de un pipeline 3D: cuántos triángulos conservar al simplificar una malla, qué nivel de detalle dibujar, cómo comprimir texturas y cómo empaquetar sus mapas UV. Sigue el artículo de Garland y Heckbert sobre el error cuádrico [17], las clases de simplificación de UCSD [8] y Princeton [3], la presentación de Nanite de Epic Games [19], la documentación de compresión por bloques de Microsoft [44], el artículo de mapas conformes por mínimos cuadrados [24] y la API de Python de Blender [40].

## 5.1 El presupuesto de rendimiento como restricción

**Definición 5.1 (Presupuesto por fotograma).** A una frecuencia objetivo de $\nu$ fotogramas por segundo corresponde un presupuesto de $1000/\nu$ ms por fotograma: 33,3 ms a 30 Hz, 16,7 ms a 60 Hz, 11,1 ms a 90 Hz (habitual en realidad virtual). Si la CPU prepara el fotograma $k{+}1$ mientras la GPU dibuja el $k$, el tiempo por fotograma es aproximadamente $\max(t_{\mathrm{CPU}}, t_{\mathrm{GPU}})$, y cada uno debe caber en el presupuesto.

El presupuesto se reparte entre sistemas (geometría, iluminación, posprocesado, física, lógica) y cada partida actúa como una restricción $\sum_j t_j(\theta) \le B$ de un problema de optimización cuya función objetivo es la calidad visual. Dos detalles importan en la práctica. Primero, la restricción debe imponerse sobre un percentil alto (por ejemplo, el 99) del tiempo por fotograma y no sobre la media, porque los tirones que percibe el jugador son los fotogramas lentos. Segundo, los costes deben medirse en el hardware de destino: el modelo *roofline* (módulo 17, §2.2) explica por qué una misma reducción de triángulos ahorra mucho cuando la GPU está limitada por el procesamiento de vértices y casi nada cuando lo está por el sombreado de píxeles.

## 5.2 Simplificación de mallas por error cuádrico

La simplificación reduce el número de triángulos colapsando aristas: los dos extremos se sustituyen por un único vértice y desaparecen las caras degeneradas. La cuestión es qué arista colapsar y dónde colocar el nuevo vértice. Garland y Heckbert [17] lo resuelven con una medida de error que se acumula sumando matrices.

**Definición 5.2 (Cuádrica de error).** Sea un plano $n^\top x + d = 0$ con $\|n\| = 1$ y sea $p = (n, d) \in \mathbb{R}^4$. Su *cuádrica fundamental* es $K_p = pp^\top$. La cuádrica de un vértice es $Q_v = \sum_p K_p$,

sumando sobre los planos de las caras que lo contienen, y el error de colocar el vértice en $v$ es $\Delta(v) = \bar{v}^\top Q \bar{v}$, con $\bar{v} = (v, 1)$. Al colapsar $(v_1, v_2)$ se toma $Q = Q_1 + Q_2$.

**Proposición 5.3 (Propiedades del error cuádrico).** *(a) $\Delta(v) = \sum_p (n_p^\top v + d_p)^2$ es la suma de los cuadrados de las distancias de $v$ a los planos. (b) $Q$ es simétrica y semidefinida positiva, y $\Delta$ es una función convexa de $v$. (c) Si $Q = \left(\begin{smallmatrix} A & b \\ b^\top & c \end{smallmatrix}\right)$ con $A \in \mathbb{R}^{3\times 3}$ invertible, el minimizador es $v^\star = -A^{-1}b$, solución del sistema lineal que se obtiene al sustituir la última fila de $Q$ por $(0, 0, 0, 1)$; $A$ es invertible si y solo si las normales de los planos generan $\mathbb{R}^3$.*

El algoritmo calcula las cuádricas iniciales, el coste $\Delta(v^\star)$ de cada arista candidata y las guarda en un montículo; en cada paso colapsa la de menor coste y actualiza las vecinas [17, §4.1]. Cada vértice guarda solo 10 números (la matriz simétrica), y el artículo informa de que una malla de 69 451 caras se simplifica en unos 15 s en el hardware de 1997. La simplificación por colapso de aristas es la que ofrecen el modificador *Decimate* de Blender en modo `COLLAPSE` [40] y la biblioteca meshoptimizer [43], y Nanite usa expresamente el error cuádrico [19]. En el cuaderno, una esfera de 20 mm de radio con 528 caras se reduce a 200 caras con una desviación máxima de los vértices de 0,30 mm, conservando $\chi = 2$.

## 5.3 Niveles de detalle y error en pantalla

Un objeto lejano ocupa pocos píxeles, y dibujarlo con todos sus triángulos desperdicia tiempo. Los *niveles de detalle* (LOD) son versiones simplificadas que se eligen según la distancia.

**Proposición 5.5 (Error proyectado).** *Con una cámara perspectiva de campo de visión vertical $\phi_v$ y una imagen de $H$ píxeles de alto, un error geométrico $\varepsilon$ a distancia $d$ ocupa aproximadamente*

$$\varepsilon_{px} = \frac{\varepsilon\, H}{2\, d\, \tan(\phi_v/2)}$$

*píxeles. En consecuencia, el nivel con error $\varepsilon$ puede usarse sin superar un umbral de $\tau$ píxeles a partir de la distancia $d^\star = \varepsilon H / \big(2\tau \tan(\phi_v/2)\big)$.*

Nanite, el sistema de geometría virtualizada de Unreal Engine 5, lleva la idea al extremo [19]: divide cada malla en grupos de 128 triángulos, los simplifica jerárquicamente con el error cuádrico en un grafo acíclico dirigido de niveles, proyecta el error de simplificación a píxeles y, en cada fotograma, elige un corte del grafo en que el error queda por debajo de aproximadamente un píxel; los triángulos diminutos se rasterizan por software. Los autores informan de unos 25 millones de triángulos rasterizados por fotograma en su demostración.

## 5.4 Selección de niveles de detalle con presupuesto

**Definición 5.6 (Mochila de elección múltiple).** Hay $G$ grupos de objetos; el grupo $g$ admite los niveles $\ell = 0, \dots, L_g$, con coste $t_{g\ell}$ (ms) y pérdida de calidad $q_{g\ell}$. Se elige un nivel por grupo, $e = (\ell_1, \dots, \ell_G)$, resolviendo

$$\min_{e} \; Q(e) = \sum_{g} q_{g\ell_g} \quad \text{sujeto a} \quad T(e) = \sum_{g} t_{g\ell_g} \le B.$$

Es un problema NP-difícil en general, pero pequeño en la práctica. La *relajación lagrangiana* elimina la restricción: para cada $\lambda \ge 0$ se resuelve $\min_e \big[Q(e) + \lambda(T(e) - B)\big]$, que se separa por

grupos (cada grupo elige el nivel que minimiza $q_{g\ell} + \lambda t_{g\ell}$). El multiplicador $\lambda$ es el «precio» de un milisegundo en unidades de calidad.

**Proposición 5.7 (Dualidad débil).** *Sea $D(\lambda) = \min_e \big[Q(e) + \lambda(T(e) - B)\big]$ y $Q^\star$ el óptimo del problema con restricción. Para todo $\lambda \geq 0$, $D(\lambda) \leq Q^\star$. Si la elección $e_\lambda$ que alcanza $D(\lambda)$ es factible y $\lambda\big(T(e_\lambda) - B\big) = 0$, entonces $e_\lambda$ es óptima.*

## 5.5 Compresión de texturas

Las GPU leen texturas comprimidas por bloques sin descomprimirlas en memoria. Los formatos BC de Direct3D [44] codifican bloques de $4 \times 4$ téxeles: BC1 usa 8 bytes por bloque (4 bits por téxel) para color RGB con alfa de un bit; BC4, 8 bytes para un canal; BC5, 16 bytes para dos canales; BC6H, 16 bytes para color HDR, y BC7, 16 bytes (8 bits por téxel) para RGB o RGBA de alta calidad, con ocho modos por bloque entre los que el codificador elige. Una cadena completa de mipmaps añade un tercio de memoria, porque $1 + \frac{1}{4} + \frac{1}{16} + \cdots = \frac{4}{3}$.

Codificar un bloque BC1 es un problema de optimización: se eligen dos colores extremos $e_0, e_1$ y, para cada téxel, un índice de dos bits que selecciona uno de los colores de la paleta $\{e_0,\ e_1,\ \frac{2}{3}e_0+\frac{1}{3}e_1,\ \frac{1}{3}e_0+\frac{2}{3}e_1\}$, minimizando el error cuadrático. Fijados los índices, el problema en $(e_0, e_1)$ es de mínimos cuadrados lineales (módulo 8, §2.2), porque cada téxel se aproxima por $(1 - w_i)e_0 + w_i e_1$ con $w_i \in \{0, \frac{1}{3}, \frac{2}{3}, 1\}$ conocido; fijados los extremos, cada índice se elige por cercanía. Alternar ambos pasos no aumenta nunca el error y termina en un número finito de pasos, como el algoritmo de las $k$-medias. La dirección inicial de $e_0e_1$ suele tomarse como la primera componente principal de los colores del bloque (módulo 3, §5.4).

## 5.6 Empaquetado UV

Para texturizar una malla se corta en *cartas* que se aplanan en el plano (parametrización) y se colocan dentro de un cuadrado de textura (empaquetado). El aplanado se plantea como minimización de una distorsión; el método LSCM de Lévy *et al.* [24] minimiza por mínimos cuadrados la desviación de la conformidad (preservación de ángulos), lo que lleva a un sistema lineal disperso. El empaquetado es una variante del problema de la mochila bidimensional: colocar rectángulos (o polígonos) sin solaparse maximizando el área útil o la densidad de téxeles uniforme, y es NP-difícil incluso para rectángulos. En la práctica se usan heurísticas voraces: por *estantes* (ordenar por altura y llenar filas), *skyline* o rectángulos máximos libres, con un margen (*padding*) entre cartas para que los mipmaps no mezclen colores de cartas vecinas. Herramientas como xatlas automatizan la segmentación, la parametrización y el empaquetado.

## 5.7 Contenido procedural y automatización con Blender

Un pipeline reproducible genera los niveles de detalle, las texturas y las comprobaciones con scripts, no a mano. Blender expone su modelo de datos en Python (`bpy`); el siguiente script crea tres niveles de detalle de cada malla de una escena con el modificador *Decimate* en modo de colapso de aristas [40]:

```
import bpy
ratios = {"LOD1": 0.5, "LOD2": 0.25, "LOD3": 0.1}
mallas = [o for o in bpy.data.objects if o.type == 'MESH']
for obj in mallas:
    for nombre, r in ratios.items():
        copia = obj.copy()
        copia.data = obj.data.copy()
        copia.name = obj.name + "_" + nombre
        bpy.context.collection.objects.link(copia)
        mod = copia.modifiers.new(name="Decimate", type='DECIMATE')
        mod.decimate_type = 'COLLAPSE'
        mod.ratio = r     # fracción de triángulos que se conserva
```

El mismo patrón sirve para el contenido procedural: un generador de rocas, edificios o vegetación es una función $G(\theta, \text{semilla})$ cuyos parámetros pueden ajustarse con optimización de caja negra contra métricas (número de triángulos, área de textura, tiempo de dibujado medido) o contra la valoración de un artista. Fijar las semillas y versionar los scripts hace que cada recurso sea reproducible, y permite integrar las comprobaciones del tema 6 en la integración continua.

| Herramienta | Para qué sirve | Licencia o tipo | Cuándo elegirla |
|---|---|---|---|
| meshoptimizer | simplificación, orden de caché de vértices, sobredibujado, cuantización | MIT | optimizar mallas para la GPU en C/C++ |
| Blender (`bpy`) | modelado, *Decimate*, scripts de pipeline | GPL-3.0 | automatizar LOD, exportaciones y comprobaciones |
| PyMeshLab | filtros de simplificación y reparación | GPL-3.0 | procesado por lotes fuera del motor |
| xatlas | segmentación, parametrización y empaquetado UV | MIT | atlas de texturas o de iluminación automáticos |
| DirectXTex | compresión BC1–BC7 y mipmaps | MIT | preparar texturas para Direct3D |
| Basis Universal | texturas supercomprimidas transcodificables | Apache-2.0 | distribuir una textura para varias plataformas |
| Unreal Engine (Nanite) | geometría virtualizada con LOD automático | comercial (licencia de Epic) | escenas con mallas de muy alta resolución |

Cuadro 5.1: Tecnologías del tema 5. Licencias según los repositorios o la documentación pública.

Tema 6

# IA generativa para 3D y validación de la geometría

Los modelos generativos producen ya objetos 3D a partir de un texto o de una imagen. Para un optimizador geométrico son a la vez una herramienta (generan candidatos o puntos de partida) y un riesgo (sus salidas pueden parecer correctas en una imagen y ser inservibles para fabricar o para un motor). Este tema describe tres familias de generadores con sus artículos originales: la destilación de un modelo de difusión 2D en DreamFusion [32], los generadores directos de funciones implícitas como Shap-E [18] y los generadores autorregresivos de mallas como MeshGPT [35]. Después formaliza la validación de la geometría para fabricación y para motores, y la integra en un bucle de generación y verificación. Se apoya en la tarea 4 del curso 16-825 de Carnegie Mellon [2] y en el filtro de fabricación aditiva de Langelaar [22].

## 6.1 Tres maneras de generar geometría

**Definición 6.1 (Muestreo por destilación de puntuación).** Sea $x = g(\theta)$ una imagen renderizada desde una vista aleatoria de una representación 3D con parámetros $\theta$ (en DreamFusion, un campo de radiancia), y sea $\hat{\epsilon}_\phi(z_t; y, t)$ la predicción del ruido de un modelo de difusión de imágenes preentrenado y congelado, condicionado por el texto $y$, sobre la versión ruidosa $z_t$ de $x$ en el instante $t$. El *muestreo por destilación de puntuación* (SDS) actualiza $\theta$ con el gradiente [32, §2.1]

$$\nabla_\theta \mathcal{L}_{\text{SDS}} = \mathbb{E}_{t,\epsilon}\Big[w(t)\big(\hat{\epsilon}_\phi(z_t; y, t) - \epsilon\big)\frac{\partial x}{\partial \theta}\Big],$$

donde $\epsilon$ es el ruido añadido y $w(t)$ una ponderación.

La idea es que el modelo de difusión «sabe» cómo debería parecer una imagen del objeto descrito, y la diferencia entre el ruido que predice y el que realmente se añadió señala en qué dirección corregir la imagen; la regla de la cadena a través del renderizador derivable (tema 4) lleva esa corrección a los parámetros 3D. DreamFusion usa el modelo base de $64 \times 64$ de Imagen sin modificarlo y un campo de tipo mip-NeRF 360, y encuentra que hace falta una guía sin clasificador muy intensa [32]. En la práctica se mezclan la predicción condicionada y la no condicionada, $\hat{\epsilon} = \hat{\epsilon}_u + s(\hat{\epsilon}_c - \hat{\epsilon}_u)$, con $s$ grande [2]. El método no necesita datos 3D, pero cada objeto es una optimización completa.

Los *generadores directos* aprenden de pares de objetos 3D y textos. Shap-E entrena primero un codificador que transforma cada objeto en los parámetros de una función implícita, que puede renderizarse como malla texturizada o como campo de radiancia, y después un modelo de difusión condicionado sobre esos parámetros; genera objetos en segundos [18]. Los *generadores autorregresivos de mallas* tratan la malla como un texto: MeshGPT aprende un vocabulario de *embeddings* geométricos de caras con un codificador de grafos y cuantización

vectorial residual, y un transformer de solo decodificador predice el siguiente índice del vocabulario, como un LLM predice el siguiente token (módulos 12 y 15) [35]. El resultado son mallas compactas, parecidas a las que haría un artista, a diferencia de las mallas densas que se obtienen extrayendo la isosuperficie de un campo; a cambio, el tamaño está limitado por el contexto (los autores entrenan con mallas de menos de 800 caras).

Ninguna de las tres familias garantiza que el resultado sea una malla cerrada, orientada, sin autointersecciones, con la escala correcta y fabricable. Esas propiedades hay que comprobarlas.

## 6.2 Validación de la geometría

**Definición 6.2 (Comprobaciones topológicas).** Sea $(V, F)$ una malla triangular. Se cuentan las *aristas dirigidas* $(a, b)$ de todas las caras (cada cara $(a, b, c)$ aporta $(a, b)$, $(b, c)$ y $(c, a)$) y las aristas no dirigidas $\{a, b\}$. La malla

(i) tiene *borde* si alguna arista no dirigida pertenece a una sola cara;

(ii) es *no variedad* si alguna arista pertenece a tres o más caras;

(iii) está *mal orientada* si alguna arista dirigida aparece más de una vez.

Una malla cerrada, variedad y orientada de forma coherente es *estanca*.

**Proposición 6.3 (Una cara invertida).** *Sea una malla estanca e invértase la orientación de una única cara $f$ cuyas tres aristas son compartidas con caras distintas de $f$. Entonces la malla sigue cerrada y sin aristas no variedad, exactamente tres aristas dirigidas aparecen dos veces, y el volumen de la proposición 1.6 disminuye en $\frac{1}{3}\det(v_a, v_b, v_c)$, el doble de la contribución original de $f$.*

Por eso una cara invertida es un defecto silencioso: no abre agujeros y una comprobación de estanqueidad basada solo en recuentos de aristas no dirigidas no la detecta, pero estropea el volumen, la iluminación (la normal apunta hacia dentro) y la clasificación interior/exterior que usan los programas de laminado.

Para *fabricación aditiva* se añaden comprobaciones geométricas:

- *Voladizos*: con dirección de fabricación $b$ unitaria, una cara de normal $n$ que mira hacia abajo necesita soporte si forma con la horizontal un ángulo menor que un umbral, habitualmente $45°$; equivale a $n \cdot b < -\cos 45° \approx -0{,}707$. Las caras apoyadas en la bandeja se excluyen. El filtro de Langelaar incorpora una restricción de este tipo dentro de la optimización de topología [22].
- *Espesor mínimo*: con la SDF de la pieza (tema 1), el espesor local es aproximadamente el doble de la distancia al borde en el eje medial; en la práctica se lanzan rayos desde la superficie hacia dentro y se mide la distancia a la cara opuesta.
- *Volumen de impresión, unidades y escala*: la caja envolvente debe caber en la máquina y estar en milímetros.
- *Cavidades cerradas*: en procesos de lecho de polvo o resina atrapan material sin consolidar.

Para un *motor de videojuegos*: número de triángulos y de materiales dentro del presupuesto (tema 5), coordenadas UV presentes y sin solapes, normales y tangentes coherentes, escala y punto de pivote convenidos, niveles de detalle generados y nombres según la convención del proyecto.

## 6.3 Generar y verificar

El esquema robusto es un bucle: el generador propone, el validador comprueba, un reparador corrige lo automático (soldar vértices duplicados, reorientar caras, cerrar agujeros pequeños) y el resultado se vuelve a validar. Si cada candidato pasa la validación con probabilidad $p$, generar varios y quedarse con el primero válido es eficaz (figura 6.1).

**Proposición 6.4 (Lo mejor de $N$).** *Si $N$ candidatos superan la validación de forma independiente con probabilidad $p \in (0,1)$, la probabilidad de obtener al menos uno válido es $1-(1-p)^N$, y para alcanzar una probabilidad $1-\delta$ basta con*

$$N \geq \frac{\ln \delta}{\ln(1-p)}.$$

El validador se convierte así en una función de recompensa verificable, análoga a las recompensas verificables con que se ajustan los LLM (módulo 21, §6.2). Y hereda su riesgo: si el generador se ajusta para pasar el validador, aprenderá a satisfacer las comprobaciones sin satisfacer la intención (por ejemplo, mallas estancas que se autointersecan si no se comprueban las autointersecciones). Es la ley de Goodhart que el módulo 21 (§6.4) estudia como sobreoptimización de la recompensa. La defensa es la misma: comprobaciones independientes que no se usan para ajustar el generador y revisión humana por muestreo.

## 6.4 Caso de estudio: validación de mallas generadas

| Herramienta | Para qué sirve | Licencia o tipo | Cuándo elegirla |
|---|---|---|---|
| trimesh | estanqueidad, orientación, volumen, reparaciones simples | MIT | validador ligero en Python |
| PyMeshLab | filtros de reparación (no variedad, agujeros) | GPL-3.0 | reparación por lotes |
| Open3D | métricas geométricas y distancias entre mallas y nubes | MIT | comparar con una referencia |
| Blender (`bpy`) | comprobaciones y correcciones dentro del pipeline del artista | GPL-3.0 | validar recursos antes de exportarlos al motor |
| Shap-E | generación de funciones implícitas a partir de texto o imagen | MIT (pesos publicados) | prototipos y generación local reproducible |

Cuadro 6.1: Tecnologías del tema 6.

# Bibliografía


[1] P. Bright, A. Edelman y S. G. Johnson. *Matrix Calculus (for Machine Learning and Beyond)*. Notas del curso MIT 18.S096, IAP 2023. §6.3 «Reverse-mode "Adjoint" Differentiation» (6.3.1–6.3.3) y cap. 4 «Finite-Difference Approximations». https://ocw.mit.edu/courses/18-s096-matrix-calculus-for-machine-learning-and-beyond-january-iap-2023/

[2] S. Tulsiani y equipo docente. 16-825: Learning for 3D Vision. Carnegie Mellon University (ediciones de 2022 a 2025). Tareas 1 «Rendering Basics with PyTorch3D», 2 «Single View to 3D» (§1 «Exploring loss functions»), 3 «Volume Rendering, NeRF, and 3D Gaussian Splatting» (§1.5 y §5.4) y 4 (§2 «Diffusion-guided Optimization»). https://github.com/learning3d

[3] T. Funkhouser. COS 526: Advanced Computer Graphics. Clase «Mesh Simplification». Princeton University, otoño de 2006. https://www.cs.princeton.edu/courses/archive/fall06/cos526/lectures/simplification.pdf

[4] L. Guibas. CS348n: Neural Representations and Generative Models for 3D Geometry. Stanford University, invierno de 2022 y primavera de 2023. Clases «Classical 3D Geometry Representations», «Generative Models I» (DeepSDF) y «Neural Functions: 3D from 2D Supervision». https://graphics.stanford.edu/courses/cs348n-22-winter/schedule.html

[5] L. Guibas y equipo docente de CS348n. Homework 2: Neural Implicit 3D Representations. Stanford University, primavera de 2023. Problema 1: (a) «Supervised SDF fitting», (b) entrenamiento eikonal, (c) *marching cubes*. https://graphics.stanford.edu/courses/cs348n-23-spring/Handouts/hmwk02.pdf

[6] F. Durand y W. Matusik. 6.837: Computer Graphics. MIT OpenCourseWare, otoño de 2012. Clases «Graphics Pipeline and Rasterization» I y II. https://ocw.mit.edu/courses/6-837-computer-graphics-fall-2012/

[7] S. G. Johnson. Adjoint methods and sensitivity analysis for recurrence relations. Notas para MIT 18.335J «Introduction to Numerical Methods», otoño de 2010 (fechadas en 2007). https://ocw.mit.edu/courses/18-335j-introduction-to-numerical-methods-fall-2010/

[8] R. Ramamoorthi. CSE 190: Advanced Computer Graphics. Clase 8 «Mesh simplification». University of California, San Diego, primavera de 2015. https://cseweb.ucsd.edu/~ravir/190/2015/lectures/190-lecture8.pdf

[9] A. Tovar. ME 60602: Topology Optimization. Programa del curso. Indiana University–Purdue University Indianapolis. Clases 8 «Sensitivity analysis», 10 «Duality and the optimality criteria method» y 12 «Well-posedness, numerical problems, filters». https://edrl.sitehost.iu.edu/courses/TopologyOptimization_Syllabus.pdf

[10] M. P. Bendsøe y O. Sigmund. *Topology Optimization: Theory, Methods, and Applications*, 2.ª ed. Springer, Berlín, 2004. Cap. 1 «Topology optimization by distribution of isotropic material».

[11] F. P. Incropera, D. P. DeWitt, T. L. Bergman y A. S. Lavine. *Fundamentals of Heat and Mass Transfer*. Wiley. Cap. 3 «One-Dimensional, Steady-State Conduction» (superficies extendidas).

[12] M. J. Kochenderfer y T. A. Wheeler. *Algorithms for Optimization*. MIT Press, 2019. Caps. «Sampling Plans», «Surrogate Models», «Probabilistic Surrogate Models» y «Surrogate Optimization».

[13] E. Andreassen, A. Clausen, M. Schevenels, B. S. Lazarov y O. Sigmund. Efficient topology optimization in MATLAB using 88 lines of code. *Structural and Multidisciplinary Optimization*, 43(1):1–16, 2011. https://www.topopt.mek.dtu.dk/apps-and-software/efficient-topology-optimization-in-matlab

[14] M. P. Bendsøe. Optimal shape design as a material distribution problem. *Structural Optimization*, 1(4):193–202, 1989.

[15] B. Bourdin. Filters in topology optimization. *International Journal for Numerical Methods in Engineering*, 50:2143–2158, 2001.

[16] Y. Hu, L. Anderson, T.-M. Li, Q. Sun, N. Carr, J. Ragan-Kelley y F. Durand. DiffTaichi: Differentiable programming for physical simulation. En *International Conference on Learning Representations (ICLR)*, 2020. https://arxiv.org/abs/1910.00935

[17] M. Garland y P. S. Heckbert. Surface simplification using quadric error metrics. En *Proceedings of SIGGRAPH 97*, ACM, 1997.

[18] H. Jun y A. Nichol. Shap-E: Generating conditional 3D implicit functions. Prepublicación, 2023. https://arxiv.org/abs/2305.02463

[19] B. Karis, R. Stubbe y G. Wihlidal. Nanite: A deep dive. Curso «Advances in Real-Time Rendering in Games», SIGGRAPH 2021. https://advances.realtimerendering.com/s2021/Karis_Nanite_SIGGRAPH_Advances_2021_final.pdf

[20] B. Kerbl, G. Kopanas, T. Leimkühler y G. Drettakis. 3D Gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 42(4), 2023. https://arxiv.org/abs/2308.04079

[21] S. Laine, J. Hellsten, T. Karras, Y. Seol, J. Lehtinen y T. Aila. Modular primitives for high-performance differentiable rendering. *ACM Transactions on Graphics*, 39(6), 2020. https://github.com/NVlabs/nvdiffrast

[22] M. Langelaar. An additive manufacturing filter for topology optimization of print-ready designs. *Structural and Multidisciplinary Optimization*, 2017. https://doi.org/10.1007/s00158-016-1522-2

[23] A. Laurentini. The visual hull concept for silhouette-based image understanding. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 16(2), 1994.

[24] B. Lévy, S. Petitjean, N. Ray y J. Maillot. Least squares conformal maps for automatic texture atlas generation. En *SIGGRAPH 2002*, ACM, 2002.

[25] Z. Li, N. B. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart y A. Anandkumar. Fourier neural operator for parametric partial differential equations. En *International Conference on Learning Representations (ICLR)*, 2021. https://arxiv.org/abs/2010.08895

[26] S. Liu, T. Li, W. Chen y H. Li. Soft Rasterizer: A differentiable renderer for image-based 3D reasoning. En *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019. https://arxiv.org/abs/1904.01786

[27] W. E. Lorensen y H. E. Cline. Marching cubes: A high resolution 3D surface construction algorithm. En *Proceedings of SIGGRAPH '87*, ACM, 1987, págs. 163–169.

[28] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi y R. Ng. NeRF: Representing scenes as neural radiance fields for view synthesis. En *European Conference on Computer Vision (ECCV)*, 2020. https://arxiv.org/abs/2003.08934

[29] T. Müller, A. Evans, C. Schied y A. Keller. Instant neural graphics primitives with a multiresolution hash encoding. *ACM Transactions on Graphics* (SIGGRAPH), 2022. https://arxiv.org/abs/2201.05989

[30] J. J. Park, P. Florence, J. Straub, R. Newcombe y S. Lovegrove. DeepSDF: Learning continuous signed distance functions for shape representation. En *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019. https://arxiv.org/abs/1901.05103

[31] T. Pfaff, M. Fortunato, A. Sanchez-Gonzalez y P. W. Battaglia. Learning mesh-based simulation with graph networks. En *International Conference on Learning Representations (ICLR)*, 2021. https://arxiv.org/abs/2010.03409

[32] B. Poole, A. Jain, J. T. Barron y B. Mildenhall. DreamFusion: Text-to-3D using 2D diffusion. En *International Conference on Learning Representations (ICLR)*, 2023. https://arxiv.org/abs/2209.14988

[33] M. Raissi, P. Perdikaris y G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.

[34] N. Ravi, J. Reizenstein, D. Novotny, T. Gordon, W.-Y. Lo, J. Johnson y G. Gkioxari. Accelerating 3D deep learning with PyTorch3D. Prepublicación, 2020. https://arxiv.org/abs/2007.08501

[35] Y. Siddiqui, A. Alliegro, A. Artemov, T. Tommasi, D. Sirigatti, V. Rosov, A. Dai y M. Nießner. MeshGPT: Generating triangle meshes with decoder-only transformers. En *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. https://arxiv.org/abs/2311.15475

[36] O. Sigmund. A 99 line topology optimization code written in Matlab. *Structural and Multidisciplinary Optimization*, 21(2):120–127, 2001.

[37] O. Sigmund y J. Petersson. Numerical instabilities in topology optimization: A survey on procedures dealing with checkerboards, mesh-dependencies and local minima. *Structural Optimization*, 16:68–75, 1998.

[38] Autodesk. Driving a lighter, more efficient future of automotive part design at GM. Artículo de Autodesk Design & Make, 1 de agosto de 2018. https://www.autodesk.com/design-make/articles/automotive-design

[39] Autodesk. Manufacturing methods in the Generative Design workspace. Ayuda de Autodesk Fusion. https://help.autodesk.com/cloudhelp/ENU/Fusion-GenerativeDesign/files/GD-MFG-METHODS.htm

[40] Blender Foundation. Blender Python API: `bpy.types.DecimateModifier`. https://docs.blender.org/api/3.4/bpy.types.DecimateModifier.html

[41] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne y Q. Zhang. JAX: composable transformations of Python+NumPy programs. Software, 2018. https://github.com/jax-ml/jax

[42] W. Jakob, S. Speierer, N. Roussel, M. Nimier-David, D. Vicini, T. Zeltner, B. Nicolet, M. Crespo, V. Leroy y Z. Zhang. Mitsuba 3 renderer. Software, 2022. https://github.com/mitsuba-renderer/mitsuba3

[43] A. Kapoulkine. meshoptimizer: mesh optimization library. Software (licencia MIT). https://github.com/zeux/meshoptimizer

[44] Microsoft. Texture block compression in Direct3D 11. Documentación de Microsoft Learn. https://learn.microsoft.com/en-us/windows/win32/direct3d11/texture-block-compression-in-direct3d-11

[45] nTop. Implicit modeling for mechanical design. Blog de nTop. https://www.ntop.com/resources/blog/implicit-modeling-for-mechanical-design/

[46] NVIDIA. NVIDIA PhysicsNeMo (antes Modulus). https://developer.nvidia.com/physicsnemo

[47] NVIDIA. Warp: a Python framework for GPU-accelerated simulation, robotics, and machine learning. Software (licencia Apache-2.0). https://github.com/NVIDIA/warp